

Lab Assignment 1 → Subm. August 1st Week

- WAP to remove duplicates from an array of doubles. In the prgm. Write a function which accepts an array of doubles & removes the duplicates.

- 1) Use loops in C.
- 2) Pointers using call by reference. & (dynam. alloc. mem.)

Linear Data Structures

- Array, lists, sets.
- One dimensional

Non-linear Data Structures

- Linked lists, trees, graphs.
- One many, many-one, many-many.

Abstract data type — behaviour of data stru. from the user ~~obvious~~ perspective.

Algorithm — 1) i/p

2) o/p

3) Clear & ~~unambiguous~~

4) Understanding

5) Finite: prgm wait to prgm.

6) Efficient

Algorithm Specifications

- 1) Comments
- 2) print(n)
- loops 3) For (i=0 to n) then {

while (i < n) then { } #

[1] Algorithm max (arr, n) :

1. result = arr[0];

2. for (i=1 to n) then {

if (arr[i] > result) then result = arr[i];

} return

3. End.

// arr[1:n]

OR arr is an array

result := arr[0]

for i := 1 to n do

if (result < arr[i]) then

result := arr[i]

}

[2] Algorithm ~~to~~ Sort (arr, n) — Bubble sort

1. For (i=0 to n-1) then {

{

if (arr[i] > arr[i+1]) then

Swap (arr[i], arr[i+1]);

}

Selection Sort

1. For (i=n-1; i>0; i--) {

max = max(arr, i);

Swap (arr[i], max(arr, i));

}

Performance Analysis:

- Time Complexity
- Space — Fixed, dynamic

⊗ Dependent quadratic loop:

```
for(i=0; i<10; i++)
  for(j=0; j<i; j++)
```

runs times
 $i=1$ 1
 $i=2$ 2
 $i=3$ 3
 $i=4$ 4
 $i=5$ 5
 $i=6$ 6
 $i=7$ 7
 $i=8$ 8
 $i=9$ 9
 $i=10$ 10
 $1+2+\dots+n \Rightarrow \frac{n(n+1)}{2}$

Asymptotic Notations

Big O: $f(n) = O(g(n))$
 order will not be worse than this.

$0 \leq f(n) \leq c \cdot g(n)$
 $c > 0, n \geq n_0$
 g = upper bound
 c = constant

Big O - upper bound	Average - Theta function	Ω - Omega function - Lower bound

$g(n) \rightarrow n$ after ignoring const. & multiple
 $2n+3$

Asymptotic Notations

1) O - Big O

$g(n) = 4n$
 $f(n) = 3n+2$
 $f(n) = O(g(n))$
 c, n_0
 $f(n) \leq c \cdot g(n)$
 $n \geq n_0$

$f(n) = O(g(n))$

$3n+2 = O(n)$

$3n+2 \leq c \cdot n$

$c, n_0 \rightarrow +ve \text{ constants / integers}$

$c=1$

$c=4$

$3n+2 \leq 4n$

$c=4$

$n=1$

$n=2$ $8 \leq 8$

$n \geq n_0$

$n \geq 2$

$10n^2+4n+2 = O(n^2)$
 $10n^2+4n+2 \leq c \cdot n^2$

$c=11$

$10n^2+4n+2 \leq 11n^2$

$n=1$ $16 \leq 11$

$n=2$ $50 \leq 44$

$n=3$ $104 \leq$

$n=4$ $178 \leq 176$

$n=5$ $272 \leq 275$

$n_0=5$

2) $f(n) = \Omega(g(n))$

$f(n) \geq c \cdot n$, c, n_0 : +ve integers such that $f(n) \geq c \cdot g(n)$

$3n+2 = \Omega(n)$

$3n+2 \geq c \cdot n$

$c=1, n_0=1$

$3n+2 \geq n$ $5 \geq 1$

time

real - total time taken for prgm.

user - time taken by prgm. in Comput.

sys - prgm + OS.

Limitations② Violations of speed from
comp. to comp.

Needs a running prgm. to begin with

Constant Time

$$i = 3 * 5$$

$$i = 315 * 9576$$

$$i = 3 + 5$$

$$i = 315 + 9576$$

$$i = 3 * 5 > 15$$

$$j = 3 * 21$$

On faster CPUs eg. 2GHz vs 1GHz, it may be left.

```

① int main() {
    int x=10, y=20, z;
    x = x+y;
    y = z*x;
    return;
}

```

Same constant time

```

② int f(int p, int q) {
    return p+q;
}
int main() {
    x=10, y=20
    z = f(x, y);
    y = x * z;
    set 0;
}

```

time prog - const.
= time(main())
+ f()

const.

① int main()

int x, y, z;

scanf("%d %d", &x, &y);

z = f(x, y);

y = z * x;

x, y, z = 10

scanf(x) - const.

For (y=0; z<x; y++)

Times: 10; z = f(x, y);

y = z * x;

return;

for library functions, time is to be told.

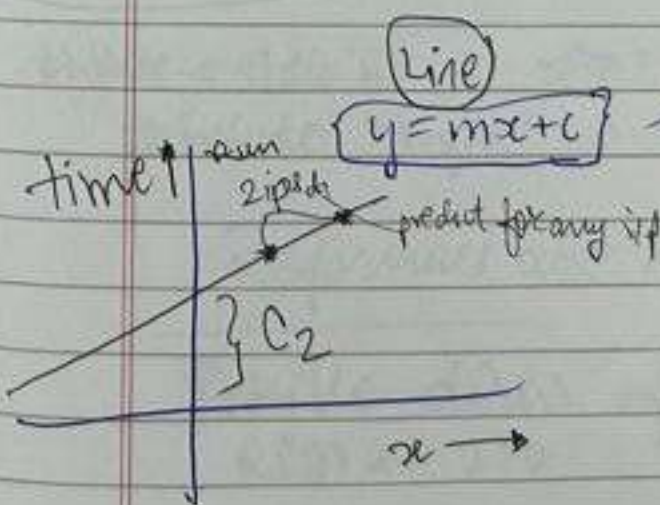
scanf printf: constant

time - taken = C
This C > previous prgm's C.

Time taken = Sum(T (various parts))

• Const time opns. $\frac{x}{z * x} = \dots$
 $y = z * x$

• Variable time opns.

• $x = 1$ opns. ($z = f(x, y)$)• $x++$ opns ($y < 10$)• $x++$ opns ($y++$)• x function calls f()• x time reqd for f()• one return, one + opn.
 $\Rightarrow$ const.• $T(x) = x * C_1 + C_2$ for one execution of var.
 $C_2 = \text{const. T.} + \text{Var. T.}$ 


```
int x, y, z = 10, k, i;
scanf("%d %d", &x, &y);
for(k=0; k<x; k++)
    for(i=0; i<y; i++)
        z = F(x, y);
y = z * x;
```

$T \rightarrow (x * y)$

$A + Bx + Cxy$

• $\text{scanf}()$ assume to be
Const. time opr. $z = 10, k, y$
 $y = z * x$

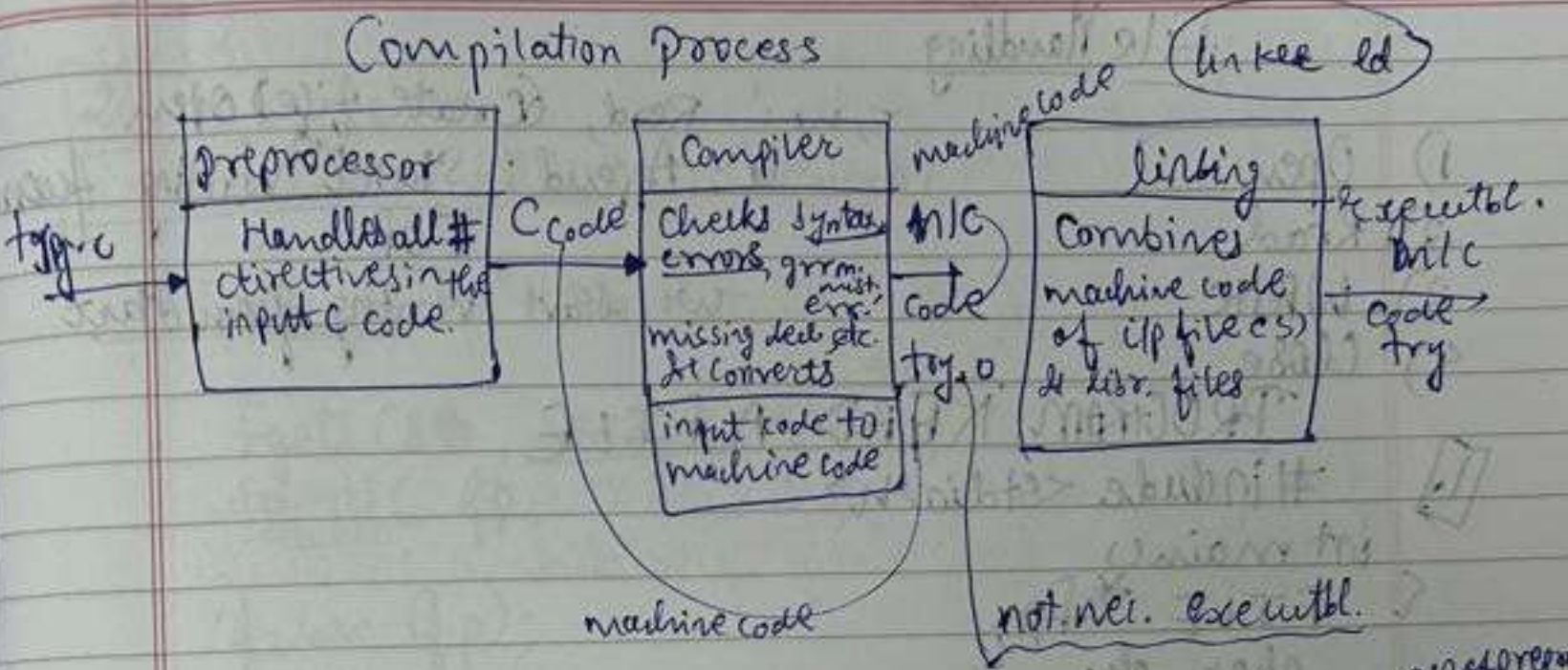
Variable time oprs.
• x Runs of loop
for $i=0$ to $k-1$
• Each run of loop

C_3 — • one $i=0$
 $y * C_2$ — • $y+1$ times $i < y$
• y times $i++$
• y times $z =$

• Each time of $F(x, y)$ is xy
• $x+1$ times $k < x$
• $T(x, y) = C_0 + x(C_1 + C_2y + C_3 + y * C_2 + C_4 + x * C_5)$

$T(x, y) = A + Bxy + Cx^2$

Compilation Process



$cc -E test.c$: Run only the preprocessor, shows the preprocessor output.
 $cc -c test.c$: Run only till compilation (No Linking).
 • Generates "test.o" file.

$cc test.o main.o try.o -o something$ — combine multiple .o files (only linking part).
 $cc add.o f.o func.o$

⊛ If there is change in only one .o file
no need to $cc -c$ other file

⊛ $cc -c f.o$
then $cc add.o f.o$ (•o = library)

$\#include <stdio.h>$: Find it in the pre defined place
 $\#include "try.h"$: Specify file at any loc.

References are managed by lang. & addresses are by program.

Possible dirns. of data transfer : RAM $\leftrightarrow$ Processor
 RAM $\leftrightarrow$ I/O

File Handling

- 1) Open
- 2) Read
- 3) Write
- 4) Close

r, w: Read, Create file & open.
a: Append: Start writing from EOF
w: Start writing from start

PROGRAM: READ A FILE

```
#include <stdio.h>
int main()
{
    FILE *fp;
    char ch;
    fp = fopen("data.txt", "r"); // file is loaded in buffer
    while(1)
    {
        ch = fgetc(fp);
        if (ch == EOF) break;
        printf("%c", ch);
    }
    OR
    while ((ch = fgetc(fp)) != EOF)
        putchar(ch);

    fclose(fp); // before closing, contents of buffer are stored in disk / destn.
    return 0;
}
```

2.

```
FILE *fp;
char s[80]; // (String)
fp = fopen("data.txt", "w");
fgetc(s, sizeof(s), stdin);
fprintf(fp, "%s", s);
fclose(fp);
return 0;
}
```

3.

~~data.txt~~ Read int values from data.txt & write sin() values of those int in another file.

```
int main()
{
    FILE *fp;
    // char
    fp = fopen("data.txt", "w");
    int i;
    for (i = 0; i < 100; i++)
    {
        fprintf(fp, "%d %f\n", i, sin(i));
    }
    fclose(fp);

    system("gnuplot -e \\"
        "set terminal png;"
        "set output 'plot.png';"
        "plot 'data.txt' with lines");

    return 0;
}
```


Q.] Compare time complexity of 2 sorting algorithms
create a set of integers varying in 1000s
a file & sort the file using 2 sorting algorithms.

Plot graph of the time taken by both the programs
using tool: gnuplot.

Compare the graphs & comment on time
complexity theoretically predicted & practically
observed.

fscanf(): Reads from file, all same: (text to binary)
fprintf(): Print text data.

Array as ADT

↳ Hides implementation

Objects: A set of pairs $\langle \text{index}, \text{value} \rangle$ where for each value of index, there is a value from the set of item.

Index is a finite ordered set of one or more dimensions.

ex: $[0, \dots, n-1]$ for 1 dimension.

$\{(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)\}$ for 2-D etc

Functions: For all $A \in \text{Array}$, $i \in \text{Index}$, $x \in \text{item}$, $j, \text{size} \in \text{integer}$

Array.create(j, list) ::= return an array of j dimension where list is j -tuple whose i th element is size of i th dimension.

Items are undefined.

Item Retrieve(A, i) ::= if ($i \in \text{index}$)
return the item associated with index value i in Array A else return error.

Array.Store(A, i, x) ::= if (i in index)
return an array with new pair $\langle i, x \rangle$ has been inserted,
else return error.

End Array

Sparse Matrix

Matrix with max no. of zero elements
Less non zero elements

1)

	0	1	2	3	4	5
0	15	0	0	22	0	-15
1	0	11	3	0	0	0
2	0	0	0	-6	0	0
3	0	0	0	0	0	0
4	91	0	0	0	0	0
5	0	0	28	0	0	0

42 elements

Non zero = 8

rows	columns	value
7	6	8
0	0	15
0	3	22
0	5	-15
1	1	11
1	2	3
2	3	-6
5	1	91
6	2	28

2)

1	2	3	4
0	5	6	7
0	0	8	9
0	0	0	10

1	0	0	0
2	5	0	0
3	6	7	0
4	8	9	10

Non-zero diagonal

upper ele. = 0 OR lower ele. = 0

Sparse Matrix

ADT Sparse matrix is

Objects: A set of triples (# Triplet form representation)
 $\langle \text{row}, \text{column}, \text{value} \rangle$
 where row & column are integers & form a unique combin. & value comes from the set items.

Functions: for all $(a, b) \in \text{Sparse matrix}$
 $x \in \text{item}$
 $i, j, \text{maxcol}, \text{maxrow} \in \text{index}$

1) sparse matrix create (maxrow, maxcolumn)

Return a sparse matrix that can hold upto max items
 $\text{max items} = \text{max row} * \text{max column}$

2) sparse matrix transpose (a) ::= Return the matrix produced by interchanging the row & column value of every triple.

3) sparse matrix add (a, b) ::= If the dimensions of a & b are same, return the matrix produced by adding corresponding items, mainly those with identical row & column values, else return error.

4) sparse matrix multiply (a, b) ::= If no. of columns in a = no. of rows in b
 Return the matrix a, produced by multiplying a by b, according to formula:

$$d[i][j] = \sum (a[i][k] * b[k][j])$$

typedef

where $d[i, j]$ is (i, j^{th}) element. ~~if~~ typedef struct
 else return error.

5) End - sparse matrix (a)

typedef struct
 {
 int row, col, value;
 } term;

void fastTranspose (term a[], term b[])

```
{
    int rowTerms[MAX_COL], start_pos[MAX_COL],
    int i, j, numCols = a[0].col;
    int numTerms = a[0].value;
    int b[0].row = numCols;
    int b[0].col = a[0].row;
    int b[0].value = numTerms;
```

if (numTerms > 0)

```
{
    for (i = 0; i < numCols; i++)
    {
        rowTerms[i] = 0;
```

```
    }
    for (i = 1; i <= numTerms; i++)
```

```
        rowTerms[a[i].col] ++;
```

```
    start_pos[0] = 1;
```

```
    for (i = 1; i <= numCols; i++)
```

```
        start_pos[i] = start_pos[i-1] + rowTerms[i-1];
```

```
    for (i = 1; i <= numTerms; i++)
```

```
        {
            j = start_pos[a[i].col] ++;
```

```
            b[j].row = a[i].col;
```

```
            b[j].col = a[i].row;
```

```
            b[j].value = a[i].value;
```

```
        } // end of loop (for)
```

```
    } // end of if st
```

```
} // end of void-func.
```


$$a = \begin{bmatrix} 15 & 0 & 0 & 22 & 0 & -15 \\ 0 & 11 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 \end{bmatrix} \quad b = \begin{bmatrix} 15 & 0 & 0 & 0 & 91 & 0 \\ 0 & 11 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 28 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
	0	3	22
	0	5	-15
	1	1	11
	1	2	3
	4	0	91
	5	2	28

Hand run

numCol = 6

numTerms = 8

b[0].row =

rowTerms[a[i].col]++

row terms

1	1	2	2	0	1
---	---	---	---	---	---

0 1 2 3 4 5

start_pos[0] = 1

for(i=1

start_pos[i]

= start_pos[i-1] + rowTerms[i-1]

j = start_pos[a[i].col] + 1

start_pos[0] ++

j = 1

b[i].row = a[i].col

b[i].col = a[i].col

b[i].value = a[i].value

O(no. of terms + no. of columns)

Transpose of sparse matrix



row	column	value
6	7	8
0	1	15
2	2	9
2	3	8
3	0	3
4	3	5
4	4	4
5	2	2
5	3	3

	0	1	2	3	4	5	6
0	0	1	0	0	0	0	0
1	0	0	0	0	0	0	0
2	0	0	9	8	0	0	0
3	3	0	0	0	0	0	0
4	0	0	0	5	4	0	0
5	0	0	2	3	0	0	0

	0	1	2	3	4	5	6
0	0	0	2	0	0	0	0
1	0	0	7	0	0	0	0
2	0	2	0	0	0	0	0
3	5	0	0	0	0	0	0
4	0	0	4	0	0	0	0
5	0	0	0	8	0	0	0

	0	1	2	3	4	5	6
0	0	3	0	0	0	0	0
1	0	7	0	0	0	0	0
2	0	2	9	8	0	0	0
3	8	0	0	0	0	0	0
4	0	0	4	5	4	0	0
5	0	0	2	12	3	0	0

6 7 12

	6	7	12
0	1	3	
1	2	7	
2	1	2	
2	2	9	
2	3	8	
3	0	8	
4	2	4	
4	3	5	
4	4	4	
5	2	2	
5	3	12	
5	4	8	

Assg 4: Initial sparse mat. Wood Feg. course
 Insert non zero value
 Addition + Addn
 Transpose # struct.

6	7	12
0	1	3
1	2	7
2	1	2
2	2	9
2	3	8
3	0	8
4	2	4
4	3	5
4	4	4
5	2	2
5	3	12
5	4	8

Write an algorithm:

Let's take a, b are matrix to be added and stored in c . Let m, n be the no. of non zero entries in a & b resp.
 Let i, j, k be index variables used for rows of a, b & c respectively.

1 → Let $i = j = k = 1$

$c[i][j] = a[i][j]$
 $c[i][j] = a[i][j] + b[i][j]$

while $(i \leq m \text{ \& \& } j \leq n)$ do

// If non value is same.

{ if $(a[i][j] == b[i][j])$ {

if $(a[i][k] == b[j][k])$ // check if column is same {

$c[k][0] = a[i][0]$

$c[k][1] = a[i][1]$

$c[k][2] = a[i][2] + b[j][2]$

$i++$; $j++$; $k++$;

else if

DS A

Course Structure

6 Units

LAB - CIE → 100 Marks

18-20 problems each lab.

Upto MSE - 10 problems

Mini project - 50m

200 to 100

CIE : 100 Marks

Grading: each assignment - 10 M

ESE → 25 M	mini proj.	2 M - attendance
ESE → 25 M	lab exam	2 M - Lab file
ESE → 50 M	Mini proj.	3 M - O/P - plagiarism/creativity
	L CIE	3 M - program

Theory

MSE → 30 marks - 3 units

TA → 10 marks - Attendance + Quiz + Class test

ESE → 60 marks - 3+3 units